R 语言基础（三）

基本运算、常用函数与逻辑运算

郑泽靖

北京师范大学统计学院

2026 年秋季学期

上讲回顾：数据结构与列表

数据结构	基本用途
向量	保存一组同类型的值
矩阵	按行和列组织同类型的值
数据框	按列保存一张数据表
列表	组合不同的 R 对象

输入代码

```
lst <- list(u = 2, v = "abcd")
lst[[1]]
names(lst) <- c("num", "string")
```

`lst[[1]]` 得到 2；更名后用 `lst$num` 提取该分量。

矩阵运算：准备数据

后面几页使用同一组矩阵 A 和 B。

输入代码

```
A <- matrix(1:4, 2, 2,
  byrow = TRUE)
A
```

Console 输出

```
     [,1] [,2]
[1,]    1    2
[2,]    3    4
```

输入代码

```
B <- matrix(
  c(2, 5, 3, 6), 2, 2)
B
```

Console 输出

```
     [,1] [,2]
[1,]    2    3
[2,]    5    6
```

同型矩阵的加减

行数和列数相同的矩阵，按对应位置进行加减。

输入代码

```
A + B
```

Console 输出

```
     [,1] [,2]
[1,]    3    5
[2,]    8   10
```

输入代码

```
A - B
```

Console 输出

```
     [,1] [,2]
[1,]   -1   -1
[2,]   -2   -2
```

例如，`A + B` 左上角的结果为 $1 + 2 = 3$。

矩阵与一个数的加减

一个数与矩阵运算时，
分别作用于矩阵中的每个元素。

输入代码

```
A + 2
```

Console 输出

```
     [,1] [,2]
[1,]    3    4
[2,]    5    6
```

输入代码

```
2 - A
```

Console 输出

```
     [,1] [,2]
[1,]    1    0
[2,]   -1   -2
```

`2 - A` 在每个位置都用 2 减去原来的元素。

按元素相乘与相除

`*` 和 `/` 对矩阵中相同位置的元素进行运算。

输入代码

```
A * B
```

Console 输出

```
     [,1] [,2]
[1,]    2    6
[2,]   15   24
```

输入代码

```
A / B
```

Console 输出

```
     [,1]      [,2]
[1,]  0.5 0.6666667
[2,]  0.6 0.6666667
```

本例两矩阵的行数、列数分别相同。

按元素进行幂运算

A^2 计算每个元素的平方。
A^B 用 B 中对应元素作为指数。

输入代码

```
A ^ 2
```

Console 输出

```
     [,1] [,2]
[1,]    1    4
[2,]    9   16
```

输入代码

```
A ^ B
```

Console 输出

```
     [,1] [,2]
[1,]    1    8
[2,]  243 4096
```

矩阵与一个数的除法

输入代码

```
C <- matrix(1:6, 2, 3,
  byrow = TRUE)
C
```

Console 输出

```
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

输入代码

```
C / 2
```

Console 输出

```
     [,1] [,2] [,3]
[1,]  0.5  1.0  1.5
[2,]  2.0  2.5  3.0
```

C / 2 将每个元素除以 2。

A 是 2×2，C 是 2×3，不能直接相加。

矩阵乘法

%*% 表示矩阵乘法。
左边矩阵的列数须等于右边矩阵的行数。

输入代码

```
A %*% C
```

Console 输出

```
     [,1] [,2] [,3]
[1,]    9   12   15
[2,]   19   26   33
```

本例：2×2 的矩阵乘 2×3 的矩阵，
得到 2×3 的矩阵。矩阵与一个数相乘使用 *。

矩阵求逆

`solve(A)` 计算矩阵 `A` 的逆矩阵。

输入代码

```
solve(A)
```

Console 输出

```
     [,1] [,2]
[1,] -2.0  1.0
[2,]  1.5 -0.5
```

- 求逆的对象须为可逆的方阵。
- 本例 `A` 可以求逆。`C` 不是方阵，不能这样求逆。

求和函数

sum() 计算所有元素之和。

输入代码

```
sum(c(1, 2, 3))
```

Console 输出

```
[1] 6
```

接下来使用矩阵 a：

输入代码

```
a <- matrix(1:6, 2, 3, byrow = TRUE)
```

$$a = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$

按行与按列求和

`rowSums()`：各行之和。

输入代码

```
rowSums(a)
```

Console 输出

```
[1]  6 15
```

第一行：$1+2+3=6$。
第二行：$4+5+6=15$。

两个函数都返回向量。

`colSums()`：各列之和。

输入代码

```
colSums(a)
```

Console 输出

```
[1] 5 7 9
```

三列分别得到 5、7、9。

数学函数作用于向量

数学函数可以逐个计算向量中的元素。
`sin()` 计算正弦值，输入角度以弧度为单位。

输入代码

```
x <- seq(1, 11, 2)
sin(x)
```

Console 输出

```
[1]  0.8414710  0.1411200 -0.9589243
[4]  0.6569866  0.4121185 -0.9999902
```

结果与 `x` 一一对应，仍是一个向量。
输出中的 `[4]` 表示该行从第 4 个元素开始。

数学函数作用于矩阵

当输入为矩阵时，`sin()` 对各位置分别计算，
并保留原来的行数和列数。

输入代码

```
x <- seq(1, 11, 2)
x <- matrix(x, 2, 3)
sin(x)
```

Console 输出

```
          [,1]       [,2]       [,3]
[1,] 0.841471 -0.9589243  0.4121185
[2,] 0.141120  0.6569866 -0.9999902
```

绘制函数曲线

在平面上绘图，先准备每个点的横坐标与纵坐标。

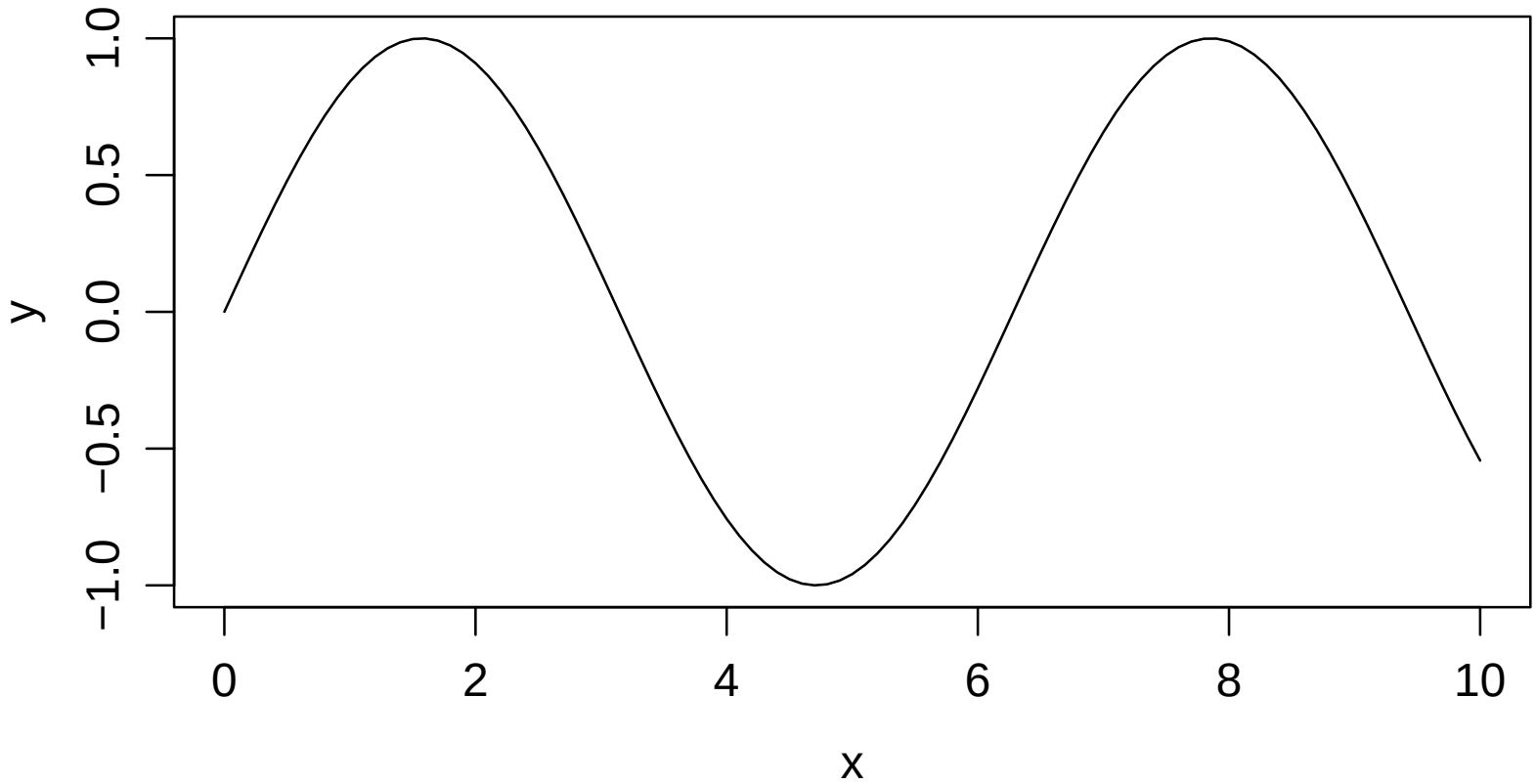
输入代码

```
x <- seq(0, 10, 0.1)
y <- sin(x)
plot(x, y, type = "l")
```

- `x`：从 0 到 10，步长为 0.1。
- `y`：各个横坐标对应的正弦值。
- `type = "l"`：用线连接这些点，字母为小写 L。

图形显示在 RStudio 的 Plots 窗格中。

正弦曲线的运行结果



曲线对应上一页的 `plot(x, y, type = "l")`。
Plots 窗格的 Export 可保存或复制图形。

课堂练习一

练习时间：9 分钟。新建脚本 `R3-practice.R`。

1. 按行创建 2×3 矩阵 `M`，填入 1 到 6。
   计算 `M + 2`。
2. 分别计算 `M` 的各行之和、各列之和。
3. 创建从 0 到 10、步长为 0.1 的向量 `x`。
   计算 `y = sin(x)`，绘制折线图。

在脚本中分段保存代码，运行后核对数值与图形。

练习一参考：矩阵运算与求和

输入代码

```
M <- matrix(1:6, 2, 3, byrow = TRUE)
M + 2
rowSums(M)
colSums(M)
```

计算	结果
`M + 2`	两行分别为 3、4、5 和 6、7、8
`rowSums(M)`	6、15
`colSums(M)`	5、7、9

练习一参考：函数与绘图

输入代码

```
x <- seq(0, 10, 0.1)
y <- sin(x)
plot(x, y, type = "l")
```

- 横坐标范围应为 0 到 10。
- 曲线应与前面的正弦曲线一致。
- x 与 y 的元素个数应相同。

第一节操作核对

操作	核对要点
`A * B`	对应位置的元素相乘
`A %*% C`	左边列数等于右边行数
`rowSums(a)`	每行得到一个和
`colSums(a)`	每列得到一个和
`sin(x)`	对各个元素计算正弦值
`plot(x, y, type = "l")`	在 Plots 窗格显示曲线

保存脚本 `R3-practice.R`。

关系运算符

关系运算判断两个值之间的关系，结果为 TRUE 或 FALSE。

运算符	含义
<	小于
<=	小于或等于
>	大于
>=	大于或等于
==	等于
!=	不等于

向量的关系运算

向量与一个数比较时，对各个元素分别判断。

输入代码

```
X <- 1:5
X <= 4
```

Console 输出

```
[1]  TRUE  TRUE  TRUE  TRUE FALSE
```

原向量 X 仍是 1、2、3、4、5。
比较结果是与 X 等长的逻辑向量。

字符的相等判断

输入代码

```
b <- "abc"
other <- b
b == "abc"
b == "ab"
b == other
```

Console 输出

```
[1] TRUE
[1] FALSE
[1] TRUE
```

字符值需要放在引号中。

逻辑运算符

运算符	含义	示例及结果
!	非	!TRUE 为 FALSE !FALSE 为 TRUE
&	与	两个条件都为真，结果才为真
|	或	至少一个条件为真，结果为真

	TRUE，TRUE	TRUE，FALSE	FALSE，FALSE
&	TRUE	FALSE	FALSE
|	TRUE	TRUE	FALSE

关系运算生成逻辑矩阵

输入代码

```
x <- matrix(1:6, 2, 3)
x
```

Console 输出

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

输入代码

```
a <- x <= 3
a
```

Console 输出

```
      [,1]  [,2]  [,3]
[1,]  TRUE  TRUE FALSE
[2,]  TRUE FALSE FALSE
```

逻辑矩阵保留原矩阵的行数、列数和位置对应关系。

逻辑矩阵：条件与取反

输入代码

```
b <- x > 2
b
```

Console 输出

```
      [,1]  [,2] [,3]
[1,] FALSE  TRUE TRUE
[2,] FALSE  TRUE TRUE
```

输入代码

```
!a
```

Console 输出

```
      [,1]  [,2] [,3]
[1,] FALSE FALSE TRUE
[2,] FALSE  TRUE TRUE
```

!a 将 a 中的真、假逐个取反。

逻辑矩阵：与、或

a 表示 x <= 3，b 表示 x > 2。

输入代码

```
a & b
```

Console 输出

```
      [,1]  [,2]  [,3]
[1,] FALSE  TRUE FALSE
[2,] FALSE FALSE FALSE
```

只有数值 3 同时满足。

输入代码

```
a | b
```

Console 输出

```
     [,1] [,2] [,3]
[1,] TRUE TRUE TRUE
[2,] TRUE TRUE TRUE
```

所有位置至少满足一个。

矩阵与一个逻辑值

单个逻辑值也可以与逻辑矩阵中的各个元素运算。

输入代码

```
a & FALSE
```

Console 输出

```
      [,1]  [,2]  [,3]
[1,] FALSE FALSE FALSE
[2,] FALSE FALSE FALSE
```

输入代码

```
a | FALSE
```

Console 输出

```
     [,1]  [,2]  [,3]
[1,] TRUE  TRUE  FALSE
[2,] TRUE FALSE FALSE
```

单个条件的短路逻辑运算

&& 与 || 用于组合单个逻辑判断。

输入代码

```
n <- 2
(n <= 3) && (n > 2)
```

Console 输出

```
[1] FALSE
```

&&：左侧为假时，
不再计算右侧。

输入代码

```
(n <= 3) || (n > 2)
```

Console 输出

```
[1] TRUE
```

||：左侧为真时，
不再计算右侧。

前面的逻辑向量、逻辑矩阵逐元素运算使用 &、|。

按条件提取向量元素

`X < 3` 先得到逻辑向量。
将它放入方括号，提取对应 `TRUE` 的位置。

输入代码

```
X <- 1:5
X[X < 3]
```

Console 输出

```
[1] 1 2
```

`X` 中小于 3 的元素为 1 和 2。

按条件修改向量元素

将 X 中小于 3 的元素统一改为 8。

输入代码

```
X <- 1:5
X[X < 3] <- 8
X
```

Console 输出

```
[1] 8 8 3 4 5
```

输入代码

```
X == 1:5
```

Console 输出

```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

组合两个筛选条件

提取大于或等于 2，并且小于或等于 4 的元素。

输入代码

```
X <- 1:5
X[(X >= 2) & (X <= 4)]
```

Console 输出

```
[1] 2 3 4
```

- 每个括号内先得到一个逻辑向量。
- & 逐位置合并两个条件。

随机抽样与 sample()

从 $1, \ldots, n$ 中等可能地取一个值，对应离散均匀分布：

$$\mathrm{P}(X = k) = \frac{1}{n}, \qquad k = 1, \ldots, n.$$

`sample()` 从给定向量中随机抽取元素。

输入代码

```
sample(x, size, replace = FALSE)
```

- `x`：供抽取的向量。
- `size`：抽取的个数。
- `replace`：是否有放回，默认 `FALSE`。

有放回与不放回抽样

输入代码

```
sample(1:10, 20, replace = TRUE)
sample(1:10, 5, replace = FALSE)
```

抽样方式	抽取个数	本例结果的特点
有放回	20	来自 1 到 10，可重复
不放回	5	来自 1 到 10，互不重复

每次运行得到的具体数值和顺序可能不同。
本例不放回时，最多只能抽取 10 个数。

模拟掷骰子

每次掷一颗均匀骰子，对应从 1 到 6 中
有放回地抽取一个数。

输入代码

```
dice <- sample(1:6, 100, replace = TRUE)
dice
```

- `dice` 保存 100 次模拟结果。
- 每个元素为 1、2、3、4、5、6 中的一个。
- 前面的结果不会使后面少一个可选点数。

运行后查看自己的结果，
无需与教材数列逐项一致。

出现次数与频率

`dice == 6` 判断每次是否出现 6。
求和时，`TRUE` 按 1、`FALSE` 按 0 计算。

输入代码

```
sum(dice == 6)
sum(dice == 6) / 100
```

- 第一行：100 次中出现 6 的次数。
- 第二行：出现 6 的次数除以总次数，即频率。

频率随模拟结果变化，不要求恰好等于 $1/6$。

硬币模拟：用 mean() 计算频率

用 1 表示正面、0 表示反面，模拟投掷一枚质地均匀的硬币 10000 次。

输入代码

```
x <- sample(c(0, 1), 10000, replace = TRUE)
mean(x)
```

`mean(x)` 计算 `x` 中数值的平均数。

- 对于 0、1 数据，平均数等于 1 出现的频率。
- 本例结果是正面出现的频率，每次可能不同，不要求恰好等于 0.5。

课堂练习二

练习时间：9 分钟。在 `R3-practice.R` 中继续。

1. 创建 `X <- 1:10`，提取大于或等于 3、且小于或等于 7 的元素。
2. 将 `X` 中小于 3 的元素改为 0，再显示 `X`。
3. 模拟掷骰子 100 次，保存为 `dice`。计算出现 6 的次数和频率。

前两题核对确定结果；
第三题核对计算方法与结果范围。

练习二参考：条件提取与修改

输入代码

```
X <- 1:10
X[(X >= 3) & (X <= 7)]
```

Console 输出

```
[1] 3 4 5 6 7
```

输入代码

```
X[X < 3] <- 0
X
```

Console 输出

```
 [1]  0  0  3  4  5  6  7  8  9 10
```

练习二参考：抽样与频率

输入代码

```
dice <- sample(1:6, 100, replace = TRUE)
sum(dice == 6)
sum(dice == 6) / 100
```

- 出现次数应为 0 到 100 之间的整数。
- 频率为出现次数除以 100，介于 0 与 1 之间。
- 同学之间的结果可能不同，计算方法应一致。

本讲操作核对

任务	主要写法
比较值是否满足条件	`<`、`<=`、`>`、`>=`、`==`、`!=`
逐元素组合条件	`!`、`&`、`|`
按条件提取、修改	`X[条件]`、`X[条件] <- 新值`
随机抽样	`sample(x, size, replace)`
计算出现频率	`sum(条件) / 总次数`

保存本讲脚本，下讲继续使用抽样与逻辑判断。